

bdelf latent_t5 / latent_vae：设计规格与实现说明

bdelf 模型文档

2026 年 8 月 30 日

摘要

本文档完整说明 bdelf 仓库中两个 Stage-1 连续 latent 模型族 `latent_t5` 与 `latent_vae` 的动机、结构、读出变体、损失函数与训练接口。二者共享 T5-small 级张量形状 ($E=512$, $d_{kv}=64$, $d_{ff}=2048$, encoder/decoder 各 6 层), 但 decoder 拓扑与辅助损失不同。本文档亦说明**为何**在 Cola 路线之外做这组对照, 二者作为扩散语言模型 (DLM) Stage-2 语义空间的初步优劣分析, 以及当前 Stage-1 长度课程的训练矩阵 (第 11 节)。工程路径、CLI 与配置表见同目录上级 `README.md`。

1 动机与定位

1.1 连续 latent 语言模型

变分自编码器 (VAE) 将观测 x 与连续潜变量 z 通过近似后验 $q_\phi(z|x)$ 与生成模型 $p_\theta(x|z)$ 联结 [1]。Cola DLM [2] 在 Stage-1 训练 Text VAE, 损失为重建、KL 正则与 BERT 式 mask 辅助项, Stage-2 再在 latent 序列上训练扩散先验。

本仓库已有官方风格的 `cola_vae` (384 维 SwiGLU 块因果 VAE)。`latent_t5` 与 `latent_vae` 是**并列实验族**: 在**相同 T5-small 维数**下对比两种 decoder 归纳偏置 (自回归 + cross-attn *vs.* 并行统一生成), 并共享 encoder 实现以便控制变量。

1.2 与仓库内其它模型的关系

- 非 HuggingFace 冻结 `t5-small` (ELF 家族用法)。`readout=e|b` 为手写 RoPE Transformer; `readout=none` 算子对齐原版 `t5-small` (相对位置偏置、RMSNorm、ReLU FFN), 但 decoder 写死双向、词表仍为 GPT-2。
- 非 `cola_vae` 替代品; Cola Stage-2 仍只加载 `cola_vae`。
- 二者注册为 `kind=latent`, 训练日志字段与 `cola_vae` 同构 (`recon_ce`、`kl`、`mask` 等)。

2 为何做这些对照实验

2.1 DLM 两阶段视角

连续 latent DLM (以 Cola 为代表 [2]) 将文本生成拆为两阶段:

1. **Stage-1 (本文档模型)**: 学习 $q_\phi(z_{1:L} | x_{1:L})$ 与 $p_\theta(x_{1:L} | z_{1:L})$, 得到逐 token 的连续表示 $z_t \in \mathbb{R}^{d_z}$;
2. **Stage-2 (DiT 先验)**: 在 $z_{1:L}$ 上训练扩散/流模型 $p_\psi(z)$, 推理时先采样 latent 再 decode 为文本。

Stage-2 默认假设: z 构成一个平滑、可扩散、语义充分的序列空间。Stage-1 的 encoder 视野、瓶颈宽度、decoder 因子分解与辅助损失, 都会改变 z 的几何与可重建性, 因而影响 DLM 上限。

2.2 相对 cola_vae 要回答的问题

cola_vae 已验证「块因果 encoder + 并行 decoder + BERT-mask」在 Cola 官方设定下可行, 但其规模 (384 维、4+4 层、SwiGLU 块) 与 DiT 默认接口已固定。本对照**不替换** Stage-2 现网路径, 而是在**可控变量**下追问:

- **Encoder 视野**: latent_t5 可选双向或单向; latent_vae 仅因果/块因果 (禁止双向)。 z_t 是否更应携带全句上下文?
- **Decoder 归纳偏置**: 自回归 + cross-attn *vs.* 并行统一生成, 何者更利于学习「可被扩散补全」的 z ?
- **Latent 维度与读出**: $z \in \mathbb{R}^E$ *vs.* $z \in \mathbb{R}^B$ ($B \ll E$), 扩散先验在低维是否更省算力、是否损失语义?
- **辅助损失形态**: span corruption (T5) *vs.* BERT-mask (VAE), 何者更能抑制 encoder 塌缩、同时保持 z 对未见 token 的可预测性?
- **骨干对齐**: 在 T5-small 张量形状下复用同一 encdec, 排除「层数/宽度不同」带来的混杂, 使对比聚焦在**拓扑与损失**。

2.3 实验产出与判据 (Stage-1)

本仓库 Stage-1 训练可直接观测: 重建 CE (recon_ce)、KL、辅助项 (mask)、以及 held-out 重建。后续若接入 Stage-2, 建议以 **latent 扩散 NLL / 生成 PPL / 前缀条件一致性** 为语义空间的外部判据; 当前文档仅给出**结构层面的先验分析** (下一节), 不预设哪条路线必然更优。

3 作为 DLM 语义空间的优劣分析

以下从「Stage-2 扩散先验 + decode」需求出发, 对两条路线作**定性**对比; z 的维数记为 $d_z \in \{B, E\}$ 。

3.1 共同前提

二者均用 Cola 式 ELBO 训练, $z_{1:L}$ 为**逐位置**连续向量序列 (非整句单向量)。扩散模型通常在 $L \times d_z$ 张量上操作; 故 d_z 与 L 共同决定先验难度。默认 $B=64$ 时 latent_vae 的 KL 在 B 维; latent_t5 (readout=b) 默认 $B=32$, KL 同在瓶颈维; readout=e 则在 $E=512$ 维施加 KL, 先验负担显著更重。readout=none 无高斯后验、无 KL。

3.2 latent_vae: 并行解码、低维 z

潜在优势

- **与 Cola 范式一致**: 并行 $p(x|z)$ 与「一次生成整句 latent 再 decode」的推理图一致; Stage-2 采样 $z_{1:L}$ 后单次前向即可得全文, 延迟低。
- **低维 $z \in \mathbb{R}^B$** : 扩散状态空间为 $L \times B$; 相同网络宽度下, 先验参数量与去噪步计算通常低于 $L \times E$ 。
- **BERT-mask 辅助**: 迫使 encoder 在输入被遮挡时仍预测语义, 减轻 z 仅编码「表面共现」、decoder 死记 x 的塌缩 [3]。
- **与块扩散/块内双向先验兼容**: 若 Stage-2 采用块级噪声 (Cola 中 block 与 DiT 对齐), 可选 $\text{block_size} > 1$ 的 encoder 与块内双向 z 更同构 (默认 $\text{block_size} = 1$ 为因果基线)。

潜在劣势

- **因果 encoder (默认)**: z_t 仅依赖 $x_{\leq t}$, 不含后文语境; 对需要全局一致性的主题/指代, z 可能欠充分, 需 decoder 从 $z_{1:t}$ 外推补全。
- **并行 decoder 与 AR 评测错位**: 训练目标为「给定全长 z 同时预测各 token」, 与自回归 PPL 度量的因子分解不一致; 生成质量需用重建或专用指标。
- **信息瓶颈**: $B=64$ 仍可能限制极细粒度语义; 若 Stage-2 需更细风格/事实, 可能表现为重建尚可但扩散样本偏模糊。

3.3 latent_t5: encoder 与 decoder self-attn 同模式

潜在优势

- **双向 (默认 $\text{bidirectional} = \text{true}$)**: encoder 与 decoder self-attn 均见全句; z_t 可编码左右语境; decoder 从 z 并行重建。作为 DLM「语义坐标」更贴近 BERT/T5 encoder 表征 [3, 4]。
- **单向消融**: $\text{bidirectional} = \text{false}$ 时 encoder 与 decoder self-attn 均为逐 token 因果, AR + cross-attn, 可与 latent_vae 的因果视野对照。
- **单向 AR 重建**: $p(x|z)$ 为自回归, 与主流 LM 评测兼容; 前缀条件时 decoder 自然续写。
- **span 辅助**: 腐蚀 encoder 输入、decoder 仍重建原句, 类似去噪自编码, 可能使 z 对局部缺失更鲁棒, 利于 Stage-2 对 z 的随机扰动。
- **readout=b**: 与 VAE 共享 $z \in \mathbb{R}^B$, 便于在**同一扩散维度**下比较「双向并行」与「因果 + 并行 / 因果 AR」谁学得更好。

潜在劣势

- **readout=e 的高维先验**: $z \in \mathbb{R}^E$ 使 Stage-2 扩散在 $L \times 512$ 空间进行, 训练与采样成本高, 且高维高斯先验与 $\mathcal{N}(0, I)$ 的 KL 更易与重建拉扯。
- **AR decoder 与并行扩散的因子分解不一致**: Stage-2 通常并行去噪各 z_t , 而 Stage-1 用 AR 学 $p(x|z)$; 若 z 各维强依赖 AR 解码顺序, 扩散「同步更新 z 」可能引入 train-test 间隙。
- **无条件生成**: 宽先验下从 $\mathcal{N}(0, I)$ 采 z 再 AR, 质量往往弱于「encode 前缀」; DLM 若强调无条件文本生成, 需依赖 Stage-2 学好 $p(z)$ 。
- **算力**: cross-attn decoder 每层较 VAE 并行 decoder 多一组 cross-attn, Stage-1 训练更贵。

3.4 对照小结

维度	latent_vae	latent_t5
z 默认维数	$B=64$	$E=512$ 或 $B=32$ (readout) ; none 无 z
Encoder 语境	因果/块因果 (禁止双向)	双向 (默认) 或单向
Decoder	并行, 一次出全长	AR + cross-attn
辅助损失	BERT-mask	span corruption
Stage-2 先验	低 ($L \times B$)	e 高; b 与 VAE 同级
难度		
推理图	采 $z \rightarrow$ 并行 decode	采/encode $z \rightarrow$ AR
与 Cola 现网	拓扑不同; 非即插	同左

实践建议(假设性) 若 Stage-2 预算有限、优先低维扩散与单次 decode, 可优先关注 latent_vae 及 latent_t5 (readout=b)。若更看重 z_t 的全句语义与 AR 下游兼容, 可优先 latent_t5 (readout=e 或 b)。最终应以 Stage-1 重建、KL-重建折中、以及接入 DiT 后的生成指标为准; 本文档不提供实验结论。

4 共享骨干：形状与算子

4.1 张量形状与默认超参 (100m.yaml)

键	latent_t5	
n_embd / d_kv / d_ff / 层数		512 / 64 / 2048 / enc6+dec6
latent_dim (B)	32	
max_seq_len		4096
tokenizer		GPT-2 (OWT 管线)
encoder / decoder self-attn	encoder: <code>bidirectional</code> ; decoder: <code>decoder_bidirectional</code>	(null= 同 en
读出	<code>readout=e b none</code>	
辅助损失	<code>lambda_span + span 腐蚀</code>	
beta_kl		0.1
kl_entropy		缺键/false 同旧哈希; 默认 YAML 开: K

与 T5-small 的 $d_{\text{model}}, d_{\text{kv}}, d_{\text{ff}}, 6+6$ 层对齐。readout=e|b 未复现 T5 相对位置偏置、RMSNorm 与 ReLU FFN, 而采用本仓库 AR 栈惯例: pre-norm LayerNorm、RoPE [5]、GELU FFN。readout=none (原 T5) 块算子对齐 HuggingFace 原版 t5-small: 相对位置偏置 (32 buckets)、T5LayerNorm (无均值、无 bias 的 RMSNorm, $\varepsilon=10^{-6}$)、DenseReluDense (ReLU、无 gate)、独立 Q, K, V, O 投影 (注意力分数不除 $\sqrt{d_{\text{kv}}}$)、基础词表 embedding 与 lm_head 权重绑定。用户写死双向: encoder 与 decoder self-attn 均为全双向, decoder 的 relative bias 亦按双向桶 (与 encoder 同构), 不使用原版 T5 的 causal decoder。

4.2 共享模块 models/latent/encdec/

SelfAttention (encdec, readout=e|b) Q, K, V 投影至 $H \cdot d_{\text{kv}}$; RoPE 施加于 Q, K 。顺序位置 $0 \dots L-1$ 切预计算表 (默认长度 4096), 超长回退按位置现场计算。

TransformerBlock pre-norm self-attn + FFN (精确 GELU, 非 tanh 近似)。dropout $p=0$ 时为 Identity。

LatentEncoder token embedding + N_{enc} 层 block; 支持三种注意力模式 (第 5 节)。

PosteriorBReadout $\mu, \log \sigma^2$ 均为 $\mathbb{R}^E \rightarrow \mathbb{R}^B$; μ 经无仿射 LayerNorm。

PosteriorEReadout 瓶颈 $\mathbb{R}^E \rightarrow \mathbb{R}^B$, 再 $\mathbb{R}^B \rightarrow \mathbb{R}^E$ 得 $\mu, \log \sigma^2$ 。

4.3 原 T5 块 (models/latent/latent_t5/t5_blocks.py)

仅 readout=none 组装。T5LayerNorm、相对位置偏置 (首层共享)、T5DenseReluDense、独立 $Q/K/V/O$ 。Decoder 每层为 self-attn $\rightarrow$ cross-attn $\rightarrow$ FFN。相对位置桶按 (Q, K, device) 缓存 (与权重无关); 无相对偏置的层不物化全零 $L \times L$ bias, 以便 SDPA 走 Flash。

5 Encoder 注意力三路

设 encoder 隐状态为 $h \in \mathbb{R}^{L \times E}$ 。

1. **双向** (`bidirectional=true`, `latent_t5` 默认): encoder 无因果 mask。Decoder 默认同为双向, 从 z 并行重建 (不 teacher-force x)。仅 `latent_t5` 可配置。
2. **逐 token 因果** (`bidirectional=false`, `block_size=1`): encoder 为 `is_causal=True`; decoder 默认同为因果 AR + cross-attn。`latent_t5` 可通过 `--set model.bidirectional=false` 选用; `latent_vae` 固定此模式 (默认 `block_size=1`)。
3. **块因果** (`block_size>1`, 可选): 块内双向、块间因果; mask 与 `cola_vae` 共用 `block_causal_mask_mod`。要求 $L \bmod \text{block_size} = 0$ 。`latent_vae` 默认 `block_size=1`, 即不启用块注意力。

`latent_t5` 另有 `decoder_bidirectional: null` 时 decoder 跟随 encoder; 显式 `false` 时 decoder 因果 AR (仅 `readout=e|b`)。`readout=none` (原 T5) **写死** encoder 与 decoder 均为双向, 禁止单向。`latent_vae` **禁止** `bidirectional=true` (配置层与实现层均拒绝; encoder 写死 `False`)。

Pad (独立 special, 非 EOS) 不进 CE / 后验平均; BERT-mask 与 span 不抽 pad、也不抽第 0 位。双向 self-attn 与 T5 cross-attn 屏蔽 pad key (`latent_t5` 始终传入 bool pad; 全 `False` 与 `None` 对注意力分数等价)。逐 token 因果 self-attn **不加** pad mask (右 pad 下真实 token 看不到右侧 pad, 以便 Flash); `block_size>1` 时块内双向叠加 pad mask。

6 瓶颈与读出

6.1 记号

后验参数化为对角高斯 $q(z|x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$, 网络输出 $\log \sigma^2$ 并 clamp 至 $[-30, 20]$ 。重参数化 $z = \mu + \sigma \odot \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$ 。KL 为

$$\text{KL}(q||p_0) = -\frac{1}{2} \mathbb{E}[1 + \log \sigma^2 - \mu^2 - e^{\log \sigma^2}], \quad (1)$$

对 z 所在空间各维取平均 (`kl_gaussian`)。

`latent_t5` 与 `latent_vae` 另有开关 `kl_entropy` (不进必填键; 缺键或 `false` 不进指纹, 与旧 YAML 同哈希)。关时后验项即为上式 KL。开 (默认 YAML) 时两项并存、同一 β :

$$\mathcal{R}(q) = \text{KL}(q||p_0) + \mathbb{E}[\log q], \quad \mathbb{E}[\log q] = -\frac{1}{2} \mathbb{E}[1 + \log 2\pi + \log \sigma^2]. \quad (2)$$

$\mathbb{E}[\log q]$ 是对角高斯熵的相反数 (不含 μ), 最小化则抬高 σ ; prior-KL 仍约束 μ 。开时 artifact tag 后缀 `-sigma`。`readout=none` 无后验, $\mathcal{R} = 0$ 。

6.2 `latent_vae`: $z \in \mathbb{R}^{L \times B}$

对 encoder 输出 h_{enc} :

$$\mu = \text{LN}(W_\mu h_{\text{enc}}), \quad W_\mu: \mathbb{R}^E \rightarrow \mathbb{R}^B, \quad (3)$$

$$\log \sigma^2 = W_\sigma h_{\text{enc}}, \quad W_\sigma: \mathbb{R}^E \rightarrow \mathbb{R}^B. \quad (4)$$

Decoder 输入为 $\text{from_latent}(z)$, $W_z: \mathbb{R}^B \rightarrow \mathbb{R}^E$, 再经与 encoder 同型的 self-attn 栈 (默认同为逐 token 因果), 一次前向得到全长 logits (非自回归)。

6.3 latent_t5: 两种读出

6.3.1 readout=e (默认): $z \in \mathbb{R}^{L \times E}$

$$b = W_{E \rightarrow B} h_{\text{enc}}, \quad (5)$$

$$\mu = \text{LN}(W_{B \rightarrow E} b), \quad \log \sigma^2 = W'_{B \rightarrow E} b. \quad (6)$$

瓶颈 $b \in \mathbb{R}^B$ 为中间压缩; 随机变量 z 仍在 E 维, 作为 decoder **cross-attention** 的 memory。Cross-attn 的 K, V 由 z 经 $\mathbb{R}^E \rightarrow H \cdot d_{\text{kv}}$ 线性投影 (标准 T5 decoder 在 E 维 memory 上的情形)。

6.3.2 readout=b: $z \in \mathbb{R}^{L \times B}$

$\mu, \log \sigma^2$ 与 latent_vae 相同 (均在 B 维)。不先将 z 扩维到 E 再作 memory; decoder 每层 cross-attn 的 K, V 为

$$K = W_K z, \quad V = W_V z, \quad W_K, W_V: \mathbb{R}^B \rightarrow H \cdot d_{\text{kv}}, \quad (7)$$

而 Q 仍来自 decoder 隐状态 $h_{\text{dec}} \in \mathbb{R}^E$ 。Self-attention 的 Q, K, V 始终在 E 维。

此变体使 T5 路径上的 KL 与 VAE 一样在瓶颈维平均, 便于公平对比瓶颈宽度; 代价是 cross-attn memory 信息带宽更低。

6.3.3 readout=none: 无瓶颈

不建 Posterior*Readout, decoder memory 即为 encoder 隐状态 $h \in \mathbb{R}^{L \times E}$ 。无重参数化、KL 记 0。encoder 与 decoder **写死双向** (并行从 h 重建); `bidirectional=false` 或 `decoder_bidirectional=false` 会报错。块算子走 `models/latent/latent_t5/t5_blocks.py` (原版 t5-small), **不走 encdec 的 RoPE 栈**。

7 Decoder 与生成

7.1 latent_t5: cross-attn decoder (self-attn 与 encoder 同模式)

每层: self-attn $\rightarrow$ cross-attn(h_{dec}, z) $\rightarrow$ FFN。readout=e|b 的 self-attn 为 RoPE (默认与 encoder 的 `bidirectional` 一致, 可被 `decoder_bidirectional` 覆盖); readout=none 的 self-attn 为 T5 相对位置偏置 (双向桶) 且写死双向。

训练

- decoder 双向: decoder 从 z 起步 (readout=e/none 时已是 E 维; readout=b 时经 $B \rightarrow E$), 一次预测全长 x 。数据仍因 `full_sequence_training=True` 传入全长。

- decoder 因果: 构造 $\tilde{x} = [\text{BOS}, x_1, \dots, x_{L-1}]$ 做教师强制 AR。不使用 GPT 式外层 `batch[:, :-1]` 移位 (否则 encoder 会丢失末 token)。

生成

- decoder 双向: 一次 decode 全长; 有前缀则 encode (前缀 + L pad) 后并行重建并覆盖前缀。无条件: VAE 读出从先验采 z ; `readout=None` 则 encode 首位 BOS + 其余 pad。
- decoder 因果 (仅 `e|b`): 有前缀则 encode 得 memory 再 AR 续写; 无条件从 $\mathcal{N}(0, I)$ 采 z 再 AR。

7.2 latent_vae: 统一生成

Decoder 仅接收 `from_latent(z)`, 与 Cola `decode_logits` 同型。无条件从先验采 $z \in \mathbb{R}^{L \times B}$ 再一次 decode; 有前缀则 encode (前缀 + L pad) 后并行 decode 并保留前缀。

8 损失函数

8.1 总目标

有瓶颈时训练态为

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \beta \mathcal{R}(q) + \lambda \mathcal{L}_{\text{aux}}, \quad (8)$$

默认 $\beta=0.1$, $\lambda=1$ (与 `cola_vae` 默认一致)。`kl_entropy` 为假 (缺键 / `false`, 不进指纹) 时 $\mathcal{R} = \text{KL}(q \parallel \mathcal{N}(0, I))$; 为真 (默认 YAML) 时

$$\mathcal{R} = \text{KL}(q \parallel \mathcal{N}(0, I)) + \mathbb{E}[\log q]. \quad (9)$$

`readout=None` 无后验, $\mathcal{R} = 0$ 。`cola_vae` 无此开关, 仍只用 prior-KL。

8.2 重建项 $\mathcal{L}_{\text{recon}}$

`latent_t5` decoder 双向 (默认): 并行 CE $-\sum_t \log p_\theta(x_t | z)$ 。decoder 因果: 教师强制 AR CE $-\sum_t \log p_\theta(x_t | x_{<t}, z)$ 。`readout=None` 时 z 即为 h 。

`latent_vae` 并行 CE: $-\sum_t \log p_\theta(x_t | z)$, 所有位置同时预测。

Held-out 评测走模型 `forward` 的重建 CE (T5 默认并行; 单向为 teacher-forced AR; VAE 并行; `config/train/eval/latent/default.yaml`)。

8.3 辅助项 $\mathcal{L}_{\text{aux}}$

`latent_vae:BERT-mask` 以概率 `mask_ratio` (默认 0.15) 将 token 换为专用 `mask_token_id=V` (V 为词表大小)。encode 被 mask 的序列, 仅在 mask 位置对原始 x 计 CE [3]。固定张量形状 (`ignore_index` 掩掉非 mask 位), 便于 `torch.compile`。无 mask 位时辅助项为 0 (不是 NaN)。

latent_t5: 原地 span CE 受 T5 span corruption 启发 [4], 但不采用变长 decoder 目标序列 (与定长训练图不兼容)。流程:

1. 按 span 级 mask 覆盖约 15% token (平均 span 长 3; 允许重叠; 不抽第 0 位); 涂色在 CPU 上完成再拷到 GPU;
2. encoder 输入将 span 换为 sentinel id $V+k$, $k \in \{0, \dots, 99\}$;
3. decoder 仍重建全长原始 x (因果时 teacher-force 为原序列右移, aux 半段重复同一输入);
4. CE 仅在腐蚀位置计算; 无 span 位时辅助项为 0。

100 个 sentinel 仅扩展 encoder embedding, 不进入 lm_head。

训练图: 清洁 / aux 沿 batch 拼接 训练态若 $\lambda > 0$, 将清洁序列与腐蚀 / BERT-mask 序列沿 batch 维拼接, 一次 encode+decode. self-attn 不跨样本, 与两次独立前向同分布。重建 CE 与后验 $\mathcal{R}$ 只用清洁半段; aux CE 只用 mask / span 位。评测 forward 不开 aux。

9 训练系统接口

9.1 CLI 与配置哈希

```
train.py --model {latent_t5|latent_vae} --config 100m-{fast|full} \
  --dataset owt --preprocess default --generate eval
```

Checkpoint: cache/checkpoints/{fast|full}/latent/{model}/{hash}/.readout、latent_dim、层数等进入 config-hash; world_size 不进哈希。

9.2 日志字段

Latent 主表: recon_ce、kl、mask、token_acc、mask_acc、beta_kl、lambda_mask (T5 的 λ 写入 lambda_mask 键, 值为 lambda_span)。

9.3 优化器

Muon 识别 attn.qkv、attn.proj、mlp.c_fc、mlp.c_proj; 读出层、from_latent、lm_head、embedding 走 AdamW。

9.4 训练循环 (与课程存盘)

train/loop.py 在 checkpoint 写完之后才预取下一步 CPU batch, 以免 curriculum_state 的 bucket 被写成下一步。非有限 loss 且权重仍有限时跳过 backward (避免 DDP 中毒); 梯度非有限则跳过 optimizer step。

10 长度课程 (Stage-1)

本节为 `latent_t5` / `latent_vae` 的**长度课程**实验协议 (非 Cola Stage-2 DiT)。与 Cola 现网 Stage-2 无关; 两条 latent 模型共用同一日程。工程命令与 dataloader 接线见 `README.md`。

10.1 预算与数据原则

全程 **10B 有效 token** (pad 不计入科学预算)。OWT 一整遍约 8-9B; 10B 约 1.1 遍, 多出的量用于长段外推与短桶回放。

- 配比按**有效 token 抽样比例**, 不是条数、不是 pad 后张量长度。
- **同一阶段不混 owt-seg512 与 owt-bucket**: 二者 512 切分规则不同, 不是同一分布。
- 同一步只含同一桶。阶段内训练图长最多两档: 短桶并入次长档 (S1/S2 钉死 512; S3 为 512 或 1024; S4 为 1024 或 2048), 微批按阶段最大 L 固定。不按桶原生四档切换 (`torch.compile` 多图与分配器碎片), 也不把短序列垫到阶段最大 L (注意力 L^2)。Pad 不计损失。

10.2 四阶段与配比

每步图 token 在阶段最大 L 时约 262K (= 峰值 $L \times$ 全局条数)。短档步按该档 L 计。

阶段	有效 token	数据集	图 L	条数	每步 pad-token
S1	3.0B	仅 owt-seg512	512	512	512^2
S2	1.5B	owt-bucket 256+512 桶	512	512	512^2
S3	2.5B	owt-bucket 256+512+1024	1024	256	256×1024
S4	3.0B	owt-bucket 四桶	2048	128	128×2048

来源	S1	S2	S3	S4	合计
owt-seg512	100%	—	—	—	3.00B
bucket 256	—	30%	10%	5%	0.85B
bucket 512	—	70%	20%	10%	1.85B
bucket 1024	—	—	70%	20%	2.35B
bucket 2048	—	—	—	65%	1.95B
阶段合计	3.00B	1.50B	2.50B	3.00B	10.00B

S1 仅在定长 seg512 上训稳重建/KL/辅助三项; S2 换 bucket 短桶切分且**不改**优化器几何 (仍 512×512 条)。S3/S4 为长度外推: 新长度桶占 70%/65%, 短桶回放递减。

10.3 优化器: 一条 WSD, 不按阶段重置

数据课程 $\neq$ 优化器课程。Stage-1 VAE 的 KL-重建折中由 Adam/Muon 动量维持; 阶段边界重置优化器易在几步内打飞后验。故全程一条 warmup-stable-decay (WSD) 轨迹, EMA 不断档:

段	有效 token	学习率
warmup (仅 S1 开头一次)	~150M	线性 $0 \rightarrow 2 \times 10^{-3}$
stable	~9.2B	恒定 2×10^{-3}
decay (仅 S4 末)	0.8B	cosine $\rightarrow 2 \times 10^{-4}$

Muon 与 AdamW 共用峰值 LR; $\beta_1=0.9$, $\beta_2=0.95$; 无 weight decay; $\beta_{kl}=0.1$, $\lambda=1$ 四阶段不变。阶段边界不改 LR、不重置动量或 EMA。若必须换 checkpoint 目录, 须整包续训并按总 10B 的 token 进度续算 WSD, 禁止本阶段 step=0 再 warmup。

S3/S4 入口同时改 L 与全局条数时, $|g|$ 尺度会短暂错配; 靠 β_2 的几十步自应消化, 不靠重置或阶梯降 LR。

10.4 阶段闸与观察窗

- **S1→S2**: dev 重建 CE 走平; KL 不塌缩、不发散; 辅助 mask_acc 明显高于随机。
- **S2→S3**: owt-bucket 的 512 桶重建不差于 S1 结束 (勿用 seg512 dev 冒充)。
- **S3→S4**: 1024 桶重建已改善; 256/512 桶无遗忘; S3 入口观察窗已过。

S3、S4 各在阶段开头留 **0.2B** 观察窗 (计入该阶段预算): 允许 CE/KL 尖峰; 窗内不降 LR、不重置优化器。S4 的 cosine 退火仅在观察窗之后的最后 0.8B 进行。评测须按桶报告 recon_ce、KL、mask_acc, 并保留 bucket-512 遗忘对照; 禁止单条 eval_loss 决策。

11 当前 Stage-1 训练矩阵

本节记录正在进行 / 即将按此清单执行的长度课程训练 (协议同第 10 节: 10B 有效 token、100m-curriculum、preprocess=latent-curriculum、OWT)。超参以 CLI --set 覆盖, 进入配置哈希; 每组独立 run 目录。 B 为 latent_dim; D 为 block_size。VAE 在 $D=1$ (逐 token 因果) 上扫全部 B ; 另在 $B=32$ 上扫块因果 $D \in \{16, 32\}$ (相对原 $D=1$ 四组**增加两组**)。T5 的 readout 为 **e** ($z \in \mathbb{R}^E$)、**b** ($z \in \mathbb{R}^B$) 或 **none** (无瓶颈); 双向 T5 无 D 。T5 有瓶颈的三组 B 一律 32 (配置默认, 不扫 16/64/128)。另增一组原 T5 (readout=none)。合计 **10** 组: VAE 6 + T5 4。

11.1 硬件

模型	服务	调度	可见 GPU	单卡
latent_vae、latent_t5	ovan-server	Slurm (cls1, QOS long)	4× RTX 4090	24 GiB

两条模型共用 ovan-server 4 卡、QOS long。VAE 两个 4 卡作业: 按 B 升序串跑 $D=1$ 四组; 对 $B=32$ 串跑 $D=16$ 再 $D=32$ 。T5 两个 4 卡作业: 单向 **e** 再 **b** 串跑; 双向默认 **e** 再原版 **none** 串跑。

11.2 latent_vae (ovan-server, 4×4090)

固定: B 读出、并行 decoder。主扫 $D=1$ (因果 encoder) 的 $B \in \{16, 32, 64, 128\}$; $B=32$ 再扫块因果 $D \in \{16, 32\}$ 。共 6 组; 提交为两个 Slurm ($D=1$ 四段、 $B=32$ 两段)。

B	16	32	64	128
$D=1$	✓	✓	✓	✓
$D=16$		✓		
$D=32$		✓		

11.3 latent_t5 (ovan-server, 4×4090)

有瓶颈三组固定 $B=32$ (配置默认)、单向 $D=1$ 。另增原 T5 一组 (无瓶颈)。提交: 单向两组一个 Slurm; 双向与原版一个 Slurm。

方向	readout	B	D	组数	合计
双向	e	32	—	1	1
单向	e	32	1	1	1
单向	b	32	1	1	1
原 T5 (双向写死)	none	—	—	1	1
					4

逐组展开:

- **双向 / readout=e**: 默认 `bidirectional=true`, $B=32$ (无 D)。
- **单向 / readout=e**: $D=1$, $B=32$ 。
- **单向 / readout=b**: $D=1$, $B=32$ 。
- **原 T5**: `readout=none`; encoder/decoder **写死双向**。无读出层、无 KL。

CLI 对应: 双向不改默认; 单向 `--set model.bidirectional=false; readout=b` 时 `--set model.readout=b`。有瓶颈组 B 用配置默认 32, 不必 `--set model.latent_dim`; 单向 D 用默认 `block_size=1`。原 T5: `--set model.readout=none` (不必也不许改方向)。

12 设计选择与局限

1. **RoPE 而非 T5 相对偏置** (仅 `readout=e|b`): 与仓库 AR/块因果路径统一。`readout=none` 已改用原版 t5-small 相对偏置 + RMSNorm + ReLU FFN; decoder 仍写死双向 (相对 bias 亦为双向桶)。
2. **GPT-2 词表**: 与 OWT 预处理一致, 不换 SentencePiece 32k。`readout=none` 将 `lm_head` 与基础 vocab embed tie (sentinel 行不进 `lm_head`); `e|b` 仍不 tie。

3. **KL 空间**: `readout=e` 时 KL 在 $E=512$; 与 `latent_vae` 默认 $B=64$ 后验容量不对等; `readout=b` ($B=32$) 可缓解; `none` 无 KL。
4. **span 辅助损失**: 固定长度「原地 CE」是工程折中, 非严格 T5 预训练目标。原 T5 消融只去掉瓶颈并写死双向, 损失仍为重建 +span 辅助。
5. **无条件生成**: VAE 读出依赖宽先验 z ; `readout=none` 无条件 encode 首位 BOS + 其余 pad。实用场景建议前缀 encode。

13 小结

`latent_t5` 与 `latent_vae` 在共享 T5-small 维数 encoder 与 Cola 式 ELBO 下, 分别检验「双向并行重建 (默认可切单向 AR) + span 辅助」与「因果 + 并行解码 + BERT-mask」两条 Stage-1 路线 (动机与 DLM 语义空间讨论见第 2-3 节)。读出变体 `readout=e|b|none` 控制随机变量所在空间与是否保留瓶颈; `none` 另将块算子对齐原版 t5-small (仍保留双向 decoder、GPT-2 词表与定长 span)。长度课程 (10B、四阶段数据配比、WSD 优化器) 见第 10 节。当前 Stage-1 训练矩阵 (二者均 ovan-server 4×4090) 见第 11 节。工程细节以代码与 `README.md` 为准; 本文档描述设计意图与完整行为规格。

References

- [1] Diederik P Kingma and Max Welling. “Auto-Encoding Variational Bayes”. In: *arXiv preprint arXiv:1312.6114* (2014).
- [2] ByteDance Research. *Continuous Latent Diffusion Language Model*. Cola DLM; arXiv:2605.06548. 2026.
- [3] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *NAACL* (2019).
- [4] Colin Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67.
- [5] Jianlin Su et al. “RoFormer: Enhanced Transformer with Rotary Position Embedding”. In: *Neurocomputing* (2024).